GenSpark

Python Foundations: Virtual Environments, History, and the Basics

This week you start building the technical foundation you'll use for the rest of the program: Python itself. This article covers environment management and the fundamentals of the language. The next articles build on it one topic at a time, until you can write your first full program, the Number Guessing Game.

Check Your Python Version

This program needs Python 3.10 or later. Open Terminal and check yours:

bash

```
python3 --version
```

If you see 3.10 or higher, you're ready. If not, install a newer version from python.org first.

Virtual Environments and .venv

The problem: different Python projects often need different versions of the same library. If you install everything globally, you'll eventually hit conflicts: project A needs version 1 of a library, project B needs version 2, and they can't both exist system-wide at once. There is a second problem on a Mac. If you installed Python with Homebrew, it protects the system installation, and installing packages globally fails with an "externally-managed-environment" error.

The solution: a virtual environment is an isolated, self-contained space for one project's dependencies. It is separate from your system's Python and from every other project's environment. Each project gets

its own, so dependencies never conflict.

Python includes a built-in tool for this called `venv`:

bash

```
python3 -m venv .venv          # create a virtual environment in a folder name
source .venv/bin/activate      # activate it
```

Once activated, packages you install with `pip install` go into that isolated environment, not your global system. You'll know it's active because your prompt shows the environment name, such as `(.venv)`. While it is active, both `python` and `python3` use the environment's Python.

bash

```
pip install requests          # installs into the active virtual environment
deactivate                    # exit the virtual environment
```

Why `.venv` as the folder name: it is a common convention, not a requirement, and most tools and `.gitignore` templates recognize it. This is why the labs ask you to choose the Python `.gitignore` template when you create a repository. You should never commit your `.venv` folder to Git. Instead, commit a `requirements.txt` file that lists the packages a project needs, so anyone can recreate the environment:

bash

```
pip freeze > requirements.txt     # save current packages to a file
pip install -r requirements.txt   # install packages from that file
```

Get in the habit of creating a fresh virtual environment for every new Python project, starting with this one.

> *Note: this can all be done in PyCharm when you create a new project. Use the power of your IDE!*

A Brief History of Python

Python was created by Guido van Rossum and first released in 1991. It was designed for readability and simplicity. It uses indentation to define code structure instead of braces or keywords, which is unusual among programming languages. This is a big reason Python code looks clean and consistent across different authors.

Python has grown into one of the most widely used languages in the world, particularly in data science, machine learning, automation, and backend web development. Its popularity comes partly from this readability and partly from an enormous ecosystem of libraries that make complex tasks accessible without writing everything from scratch.

You'll use Python throughout this program, including in the agentic AI and RAG modules later on, so building real fluency now pays off repeatedly.

Running Python Code

There are two ways to run Python.

Interactive mode. Type `python3` in your terminal and you get a prompt (`>>>`) where you can try one line at a time. Type `exit()` to leave. This is useful for quick experiments.

Script mode. Save your code in a file that ends in `.py` , then run it:

bash

```
python3 hello.py
```

In this program you'll mostly run scripts, because a script is saved, repeatable, and can be committed to Git.

Variables and Data Types

A variable is a name that refers to a value stored in memory. Python doesn't require you to declare a variable's type. It figures out the type from what you assign.

python

```
name = "Alex"          # string (text)
age = 28               # integer (whole number)
```

```
height = 5.9            # float (decimal number)
is_active = True        # boolean (True/False)
```

The core built-in types you'll use constantly:

- **String (`str`):** text, written in quotes: `"hello"`
- **Integer (`int`):** whole numbers: `42`
- **Float (`float`):** decimal numbers: `3.14`
- **Boolean (`bool`):** `True` or `False` , used for logic and conditions

You can check a variable's type with the built-in `type()` function:

python

```
print(type(age))    # <class 'int'>
```

Notice that `"28"` in quotes is a string, while `28` is an integer. They look alike but behave differently, and mixing them up is one of the most common beginner errors.

Naming Variables

- Names can contain letters, digits, and underscores. They cannot start with a digit or contain spaces.
- Names are case sensitive: `score` and `Score` are two different variables.
- Use `snake_case` for variables: lowercase words joined by underscores, such as `secret_number` or `attempts_used` .
- Use `ALL_CAPS` for constants, which are values that don't change while the program runs, such as `TIME_LIMIT = 60` .
- Choose names that describe the value. `attempts_used` is clear. `x` is not.

Math and Updating Variables

Python does arithmetic with these operators:

Operator	Meaning	Example	Result
`+`	Add	`7 + 2`	`9`
`-`	Subtract	`7 - 2`	`5`
`*`	Multiply	`7 * 2`	`14`
`/`	Divide (always gives a float)	`7 / 2`	`3.5`
`//`	Divide and drop the remainder	`7 // 2`	`3`
`%`	Remainder	`7 % 2`	`1`
`**`	Power	`7 ** 2`	`49`

To change a variable, assign a new value to it. A common pattern is counting:

python

```
attempts_used = 0
attempts_used = attempts_used + 1   # now 1
attempts_used += 1                  # shorthand for the line above, now 2
```

The same shorthand works with `-=`, `*=`, and `/=`. You'll compare values with operators like `<` and `==` in the article on decisions.

Comments

Comments are notes in your code that Python ignores when running the program. They are for humans, not the computer. As covered in the coding best practices article, good comments explain why, not what.

python

```
# This is a comment on its own line
attempts_used = 0   # a comment can also follow code
```

Python has no multi-line comment. To write a longer note, start each line with `#`. A string in triple quotes placed just under a function's first line is called a docstring. You'll meet docstrings when you learn functions.

Print Statements

`print()` shows text on the screen. It is the most basic way to see what your program is doing, and you'll use it constantly, especially while debugging.

python

```
print("Hello, world!")
print(name)
print("Age:", age)                    # commas add a space between items
print(f"Name: {name}, Age: {age}")    # f-string: embeds variables in a st
```

The f-string (a string that starts with `f` and uses `{}` around variables) is the modern, preferred way to build text that includes values. You'll use it constantly.

Input and Output

Output is data your program produces, most often with `print()`. Input is data your program receives from a user, a file, or another system. The next article covers the `input()` function and converting what the user types into numbers. File input and output comes in Week 3.

Why This Matters for an FDE

Nearly everything you build in this program is written in Python, from the Number Guessing Game and Tic-Tac-Toe in the coming weeks to the RAG pipelines and agentic systems later on. Getting comfortable with these fundamentals now, instead of rushing through them, is what makes the advanced weeks manageable instead of overwhelming.

Comprehension Check

1. Why shouldn't you commit a `.venv` folder to Git, and what file should you commit instead so others can recreate your environment?
2. What data type is each of these: `count = 10`, `ratio = 0.5`, `label = "10"`? Why is the third one different from the first?
3. Write an f-string that prints a variable called `city` inside the sentence "I live in ___."
4. If `score = 5`, what is `score` after running `score += 2`?

← Previous: Linux/CLI; Docker basics; CI/CD Introduction

Next: Input, Output, Type Conversion, and Checking Text →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!